Chapter route

How can services remain understandable and recoverable without shared memory, synchronized clocks, or failure-free nodes?

Prerequisites are network stacks, RPC, processes, and filesystem interfaces.

- Compare network and distributed operating systems.
- Analyze naming, routing, RPC, and failure semantics.
- Check DFS cache consistency and availability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

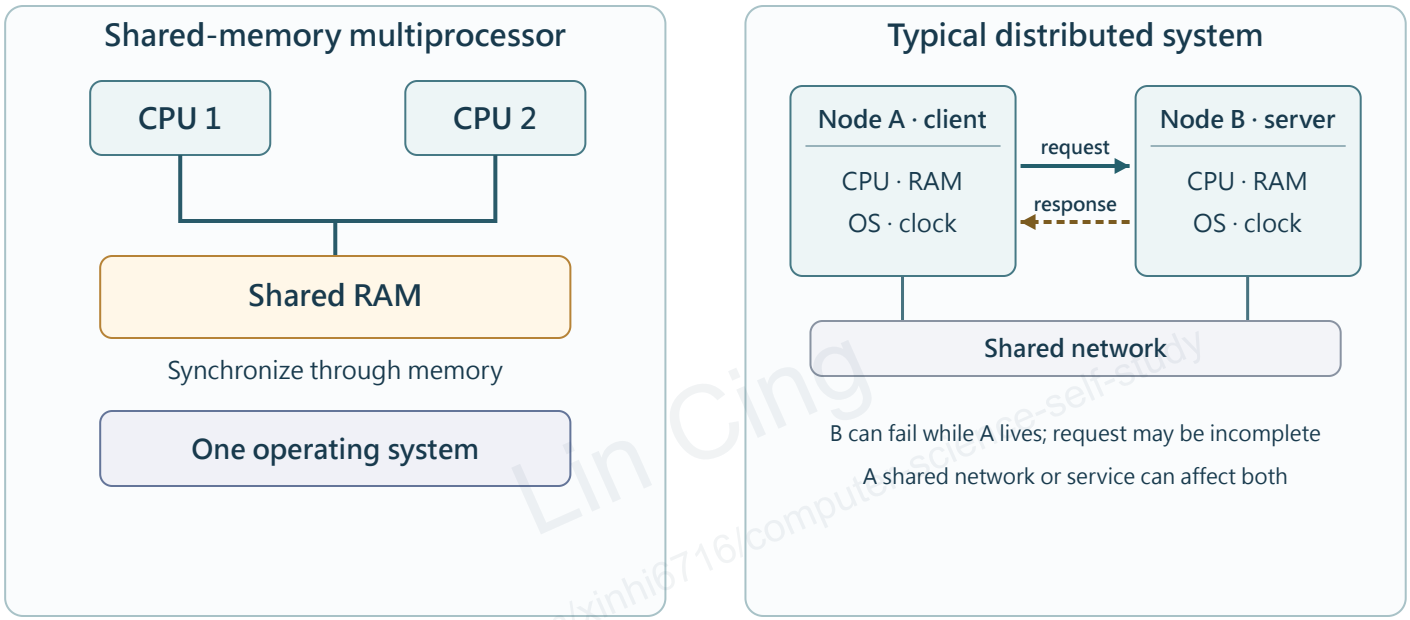
Message passing and its limits

In this chapter's typical message-passing model, nodes have local memory and OS instances and communicate by messages; no shared physical memory or perfect global clock is assumed. Distributed shared memory can provide an abstraction above this model, and synchronized clocks still have error. Local and remote are relative to the requester. Sharing, speedup, reliability and communication are goals, not automatic consequences of distribution.

For a fixed 100 s job, 20 s is serial, 80 s divides evenly across four nodes, and coordination costs 5 s. $T_4=20+80/4+5=45$ s, so speedup is $100/45\approx2.222$. Ignoring coordination, the infinite-node Amdahl bound is $100/20=5$. More nodes cannot remove serial work and can increase coordination costs.

At **one fixed read instant**, three interchangeable replicas have independent unavailability events, each with probability $p=.01$. If networking and metadata are available and any available replica can return valid data, the probability of an unavailable read is $p^3=10^{-6}$. Shared power, rack or a required metadata leader can invalidate independence. Each replica having failed at some point during an interval does not imply that all were unavailable simultaneously.

Shared memory and message passing



Models only: performance and reliability depend on load and failure domains.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shared memory and message passing (continued)

Processors coordinate through shared memory in the first model. In the second, nodes exchange requests and responses across a network. A client may not know whether its request executed after a server failure or network interruption. Shared services and networks can fail together; distributed shared memory is another possible abstraction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transparency and machine boundaries

Network OSs expose remote hosts/copies explicitly. Distributed OSs/middleware can unify naming, access, and migration. Transparency packages location/invocation and failure semantics; it does not remove physical latency/failure.

Remote login executes a remote session; transfer creates another file copy; DFS provides shared remote-file behavior. SSH/SFTP still need host authentication/user authorization; ordinary FTP is not inherently encrypted. Anonymous services need constrained namespaces/operations. Naming, identity mapping, network faults, and conflicting copies require rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Moving data or computation

For 1 GiB input, 1 MiB result, 100 MiB/s throughput, and 20 ms RTT, moving all input costs at least 1024/100+.020=10.260 s before local compute. Sending computation to a remote CPU taking .500 s and returning 1 MiB takes .500+.010+.020=.530 s, with negligible request. Include CPU queues, startup, serialization, and output size before generalizing.

Whole-file caching pays upfront for later local access. On-demand caching transfers requested blocks. Reading only 1 MiB makes a whole 1 GiB transfer waste 1023 MiB; reading all blocks through unpipelined RTT-bound RPCs can instead favor bulk transfer.

Process migration moves/reconstructs PC, registers, memory, open files/sockets, and environment. Sending code for fresh remote execution is computation placement, not migration of running state. Cross-ISA/ABI execution needs compatibility rather than copying register bits blindly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Scope and latency components

LANs commonly cover smaller administrative/geographic areas; WANs connect sites. Capacity/latency depend on links, distance, queues, load. Switches, routers, and APs have different roles. Switched Ethernet is not universally one shared collision bus; a congested LAN can underperform a high-capacity WAN.

Latency $\approx L/R$+propagation+queue+processing. A 1500 B packet on 1 Gbit/s serializes in 12 µs. One-way 5 ms propagation dominates. A tenfold rate increase reduces serialization to 1.2 µs without dividing propagation by ten.

100 Mbit/s equals 12.5 decimal MB/s before headers; MiB/s uses 2^{20} B. Wireless nominal rates differ from goodput. These are explicit models rather than current product maxima.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study/

Bandwidth-delay product

100 Mbit/s and 40 ms RTT give BDP=4×10^6 bits=500000 B$\approx$488.28 KiB. A 64 KiB outstanding-data limit bounds throughput at 65536 • 8/.040=13.1072 Mbit/s despite 100 Mbit/s capacity.

This assumes no loss, fixed RTT, and no other bottleneck. TCP congestion/receive windows, ACK behavior, slow start, retransmission further constrain results. Send buffers remove at most one bound, not congestion/server CPU. A 16 B RPC can be dominated by RTT rather than serialization, explaining the importance of placement/cache.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Names, addresses, ports, and objects

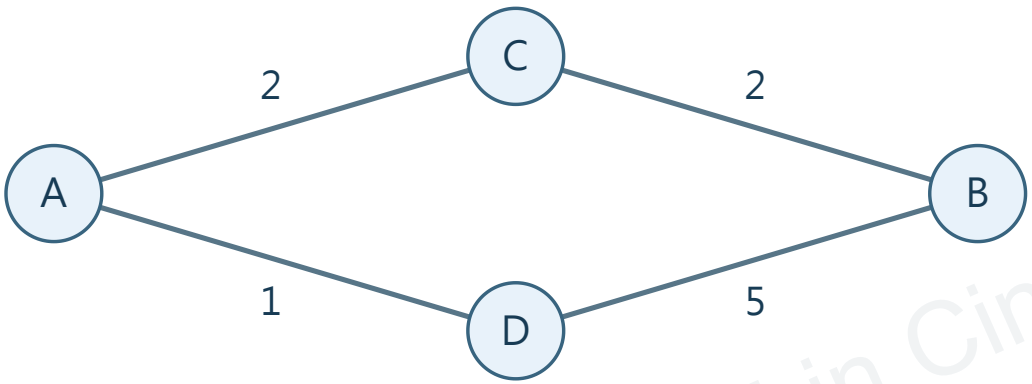
`store.lab.test` is a domain name, potentially resolving to IPv4 such as 198.51.100.20 or to IPv6. An IP address is not a user or a permanent machine identity. A service also involves transport protocol, port and application naming. A TCP connection is typically distinguished by source/destination addresses and ports and protocol; a port is not a PID.

DNS uses resolvers, queries, caches and delegation; a label does not require its own server. Assume root delegates test and test delegates lab.test. On cache miss, the resolver follows known-root referrals to an authoritative A record. This example assumes a resolver that obeys TTL, with no serve-stale behavior or local override; that is not a universal implementation rule.

At t=0 an A record .20 has TTL 600 s; at t=100, 500 s remains. A change at the authority to .30 at t=200 need not replace the cached .20 before t=600. After expiry, a successful new query obtains .30 if the authoritative record has not changed again. Failure to query does not make a stale value a verified new answer. DNS integrity and endpoint authentication need corresponding mechanisms.

Routing on a fixed graph

One nonnegative weighted graph: every distance update



From A: D=1, C=2, B=∞

Settle D: B=1+5=6

Settle C: B=min(6,2+2)=4

Settle B: A → C → B, cost4

If C–B fails, recompute A→D→B, cost6. Dynamic protocols need detection/convergence time.

Edges are A–C=2, C–B=2, A–D=1, D–B=5. Dijkstra starts with d(D)=1, d(C)=2, d(B)=∞. Settling D gives d(B)=6; settling C lowers it to 4; settling B yields A→C→B, cost 4. Losing C–B gives A→D→B, cost 6 after recomputation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Routing on a fixed graph (continued)

Static routing uses configured/admin rules; virtual circuits select session paths; dynamic protocols update metrics/reachability. Dynamic does not mean every packet chooses the emptiest link or converges without loops/drops. A path preserves order only with suitable link/queue behavior; transport supplies end-to-end stream ordering.

Control plane computes routes; data plane forwards from tables. SDN can separate them but still needs consistent updates, failure handling, and protected management.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Switching and connections

Frames, packets, and datagrams have layer-specific headers/limits and can vary in length. Application messages can span packets or share a TCP read. TCP reserves no dedicated physical circuit; applications can implement reliability over UDP.

Circuit switching reserves path resources; message switching stores/forwards whole messages; packet switching pipelines fragments with statistical multiplexing. Across two 1 Mbit/s links, ignoring headers/propagation/queues, a 4000 B message takes 64 ms. Four 1000 B packets each take 8 ms/hop; pipeline completion is $(2+4-1) \cdot 8 = 40$ ms.

With 40 B per-packet headers, it becomes $5 \cdot 1040 \cdot 8/10^6 = 41.6$ ms. One 40 B whole-message header gives 64.64 ms. These are store-and-forward assumptions, not all cut-through/circuit networks.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Layer responsibilities

Physical carries signals/bits; link carries frames; network addresses/forwards; transport serves process endpoints; session manages dialogue; presentation encodes representations; application defines service operations. TCP/IP often places session/presentation in libraries/applications. Fewer named layers does not prove efficiency.

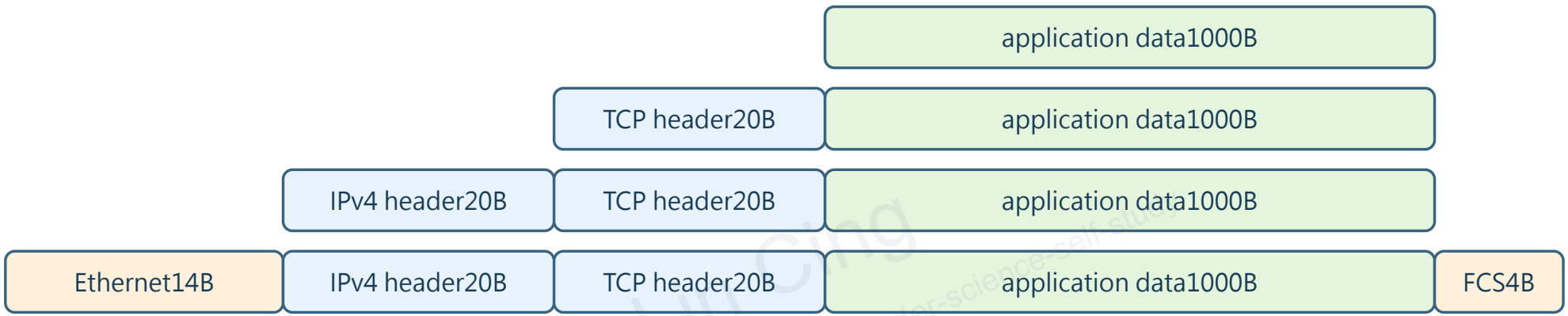
TCP offers ordered reliable bytes and flow/congestion control without message boundaries. UDP preserves datagrams without equivalent built-in delivery/order. IP is best effort. NIC offload can move checksums/segmentation into hardware, so no universal three-hardware/four-software split exists.

Serialization specifies lengths, byte order, encoding, versions. Raw C structs with padding/pointers are not portable protocols.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Encapsulation length accounting

Each layer adds headers/trailers; account for every length



Widths are not to scale. IP total1040 B; MAC frame1058 B. Preamble/SFD and IFG are separate.

1000 B application data plus 20 B TCP gives 1020 B segment; 20 B IPv4 gives 1040 B IP total. Ethernet II 14 B header+1040+4 FCS=1058 B. Add 8 B preamble/SFD and specified 12 byte-times IFG for 1078 wire byte-times.

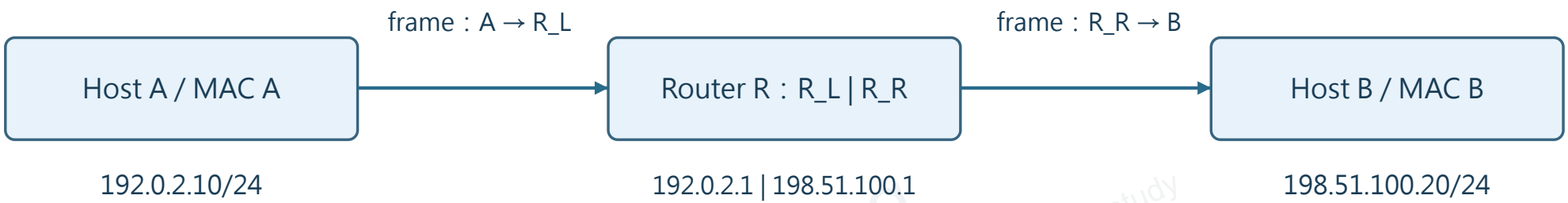
Encapsulation length accounting (continued)

With IPv4 MTU 1500, this fits one frame. MSS subtracts TCP header from available IP payload; 1500 is not all application data. VLAN/options/tunnels/security change accounting. TLS endpoints are not every forwarding router. A lower checksum detects accidental corruption rather than authenticating malicious changes with a secret key.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two Ethernet LANs

Across LANs without NAT: each hop uses different MAC addresses



A ARPs next-hop 192.0.2.1, not remote B on the left LAN.

IP stays192.0.2.10 → 198.51.100.20. Router decrements TTL64→63; recompute checksum/FCS.

No NAT/tunnel. A=192.0.2.10/24, MAC A. Router left=192.0.2.1, MAC R_L; right=198.51.100.1, MAC R_R. B=198.51.100.20/24, MAC B. DNS gives B; /24 comparison selects next-hop 192.0.2.1.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Two Ethernet LANs (continued)

Without cached R_L, A broadcasts ARP for 192.0.2.1 on its LAN. R answers. Frame 1 has src=A, dst=R_L, IP src=A, dst=B. A does not ARP remote B directly. Router looks up route, decrements TTL 64→63, updates IPv4 checksum, resolves B on the right LAN, and sends frame 2 src=R_R, dst=B with unchanged A→B IP endpoints.

B decapsulates and demultiplexes by transport port/connection. Routers normally do not forward ARP broadcasts across L2 domains; ARP replies do not cryptographically prove identity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Header changes and minimum frames

Each hop rebuilds Ethernet addresses/FCS. IPv4 TTL/checksum change; source/destination IP, TCP endpoints, payload remain under the stated exclusions. Saying only Ethernet changes is false. TTL reaching zero prevents normal forwarding; error reports can themselves be lost/filtered.

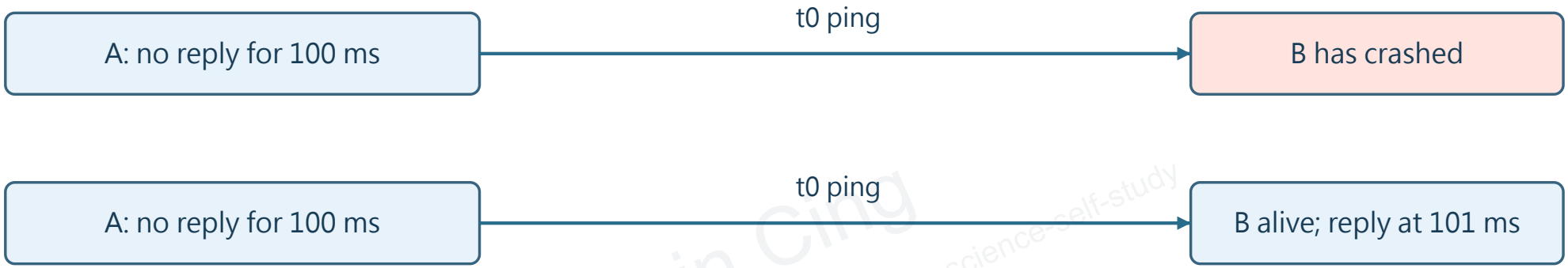
Untagged Ethernet II has 6 B destination, 6 B source, 2 B EtherType, 46–1500 B data/pad, 4 B FCS, minimum 64 B from destination through FCS. Padding is not included in IP total length; preamble/SFD is outside that frame denominator.

A 4 B TCP payload plus 20+20 headers is 44 B IP, requiring 2 B pad. Frame 14+46+4=64 B. Add 8 B preamble/SFD and 96-bit IFG=12 byte-times to 84. At 100 Mbit/s, it occupies 6.72 µs. IFG is idle time, not 12 bytes appended to IP; high-speed PHY tolerances are outside this model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Failure-detector indistinguishability

Same local observation, different remote states



A has the same history through 100 ms. Without delay bounds, timeout does not prove crash.

In an asynchronous system without known message or scheduling delay bounds, A pings at t=0 and sees no reply by 100 ms. B may have crashed, a link may be partitioned, a message may be lost, or B may simply be slow. Compare B crashed with B alive but replying at 101 ms: A's local history through 100 ms is identical. An algorithm declaring a crash solely from that observation errs in the second execution.

Failure-detector indistinguishability (continued)

For a bounded teaching model, B remains operational; heartbeat **nominal send times** are H=1 s apart, each actual send occurs 0–s after its nominal time, with s=.2 s; every message arrives with one-way delay in 0–δ, δ=.1 s. Receiver processing adds no delay and its elapsed-time clock has no drift. The previous heartbeat can arrive immediately at its nominal time and the next no later than H+s+δ=1.3 s after it. A 1.5 s timeout measured from the last arrival does not falsely suspect this live node under those assumptions. Loss, partition, an exceeded delay bound or clock error invalidates this guarantee.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

New leaders need old-leader fencing

A writes shared storage. After a partition B times out and declares itself leader while A still writes, creating split brain. Heartbeats and local declarations alone do not guarantee exclusion.

In the teaching fencing model, a correct coordinator issues monotonic epochs and storage installs currentEpoch=43 before B operates. **Each write's token check and commit must be atomic with respect to epoch installation, or equivalently serialized in the same linearization order.** A write with epoch 42 that linearized before 43 was installed belongs to the old term; storage must reject one ordered after installation. Checking once before commit is insufficient: an old write can pass under 42, be delayed, and otherwise reach storage after 43 is installed.

Every write path must enforce the token and issuance and installation must be correct; partitions cannot independently increment numbers and call that legitimate ownership. Confirmed hardware or access isolation can also fence old nodes. Before A rejoins as a follower, it synchronizes committed state, membership/epoch and unfinished requests. IDs or generations prevent stale callbacks and replies from matching new requests after restart.

Lost RPC replies and deduplication

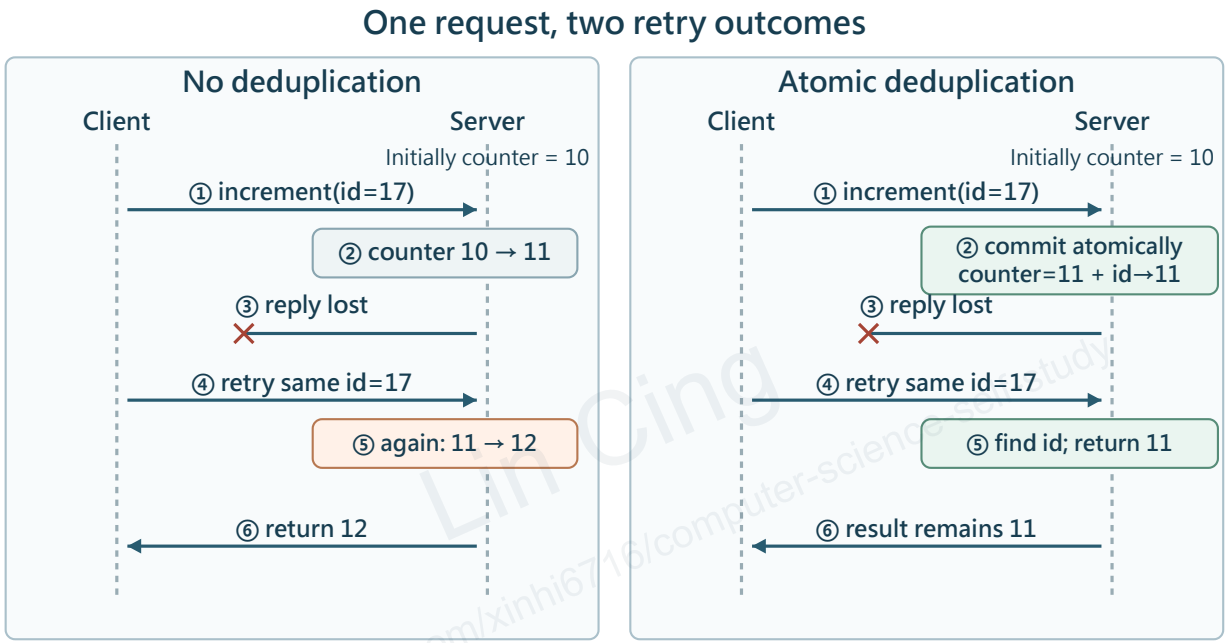
Counter 10 receives increment(id17), commits 11, and loses reply. Naive retry yields 12. TCP retransmission supplies connection-byte delivery, not cross-reconnection application deduplication.

In the explicit at-most-once model, a server transaction checks durable table[id]. Existing IDs return saved results; new IDs atomically persist counter++ and table[id]=result before replying. Retrying id 17 returns 11. Updating counter before recording ID permits duplicate increment after crash; recording ID first permits lost effect.

The proof requires shared transaction/recovery scope, unique IDs, and retention long enough for retries. Permanent disconnection can still leave the client uncertain. External email/payment outside the transaction needs another protocol.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RPC retries: lost replies and deduplication



ID must be unique and retained throughout retries; external effects need a separate protocol.

Persist the counter update and `id→result` in one transaction. Retrying with the same unique ID returns the recorded result while its record remains available. External effects such as email or payment require a separate protocol.

Fault domains and failover capacity

Two active servers each handle 100/s with total load 160/s. Losing one leaves 100/s and at least 60/s unsustainable. Replicas being able to take over does not preserve SLO without spare capacity, shedding, or degradation.

Stateless worker retries can still duplicate effects. Stateful services additionally need logs, recovery, replication, locks. RAID addresses selected disk faults, not cross-site availability or backups of logical corruption. Recover durable committed state, reconcile in-flight operations, then serve. Missing commit reply does not imply uncommitted work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Observable transparency and scaling

Location transparency hides placement;migration transparency preserves naming/operations after movement;replication transparency hides copies. Failure transparency cannot indefinitely hide a nonreturning operation. User mobility also needs identity, environment, and session consistency.

Ten workers at 100/s offer 1000/s, but a one-core coordinator spending 2 ms/request caps the system at 500/s. An eleventh worker does not lift it. Caching, batching, or partitioning can alter the bottleneck's per-request demand. Replication/rebuild traffic consumes capacity too.

Hadoop 1.2.1 HDFS NameNode manages namespace/block locations; DataNodes store/transfer blocks; client data bypasses NameNode. Large blocks, streaming, and a simplified write model favor bulk data rather than full POSIX sharing or arbitrary low-latency transactions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

HDFS payload and flow

Custom 64 MiB blocks and replication 3 split 130 MiB into 64, 64, 2. Logical 130 MiB yields 390 MiB replicated payload, excluding metadata, checksums, padding/allocation. The last 2 MiB need not transfer a full 64 MiB of payload but still needs block location metadata.

Client gets block/DataNode locations from NameNode, then reads DataNodes directly; writes can use a DataNode pipeline. Heartbeats/block reports detect missing replicas and trigger re-replication with delay/network/disk cost.

Secondary NameNode assists checkpointing;it is not an automatic hot standby. Fault-domain-aware placement helps data availability, while metadata and client reachability remain required.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Names, stable identities, and locations

Location transparency hides physical placement;independence preserves names after movement. ServerA:/x exposes server location. Client mounts can hide it differently per client;global namespaces add common naming/management.

Path /course/a.txt→FID(volume7, file 12, generation 3), volume7→serverA. Moving to serverB changes only the second map. Reusing deleted file 12 must change generation or stale FIDs can identify the new file. Path→FID and volume→location caches have different invalidation conditions.

Replicated/cached location maps still need versions/coherence. Indirection does not eliminate updates. Diskless boot requires firmware/initial protocols to load kernel before DFS exists; an unloaded DFS cannot load itself.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete cache-cost accounting

100 reads of 4 KiB at 5 ms remote cost total 500 ms uncached. Ten misses at 5 plus 90 hits at .1 ms cost 59 ms and 40 KiB network payload versus 400 KiB. This specified workload does not prove a universal hit ratio.

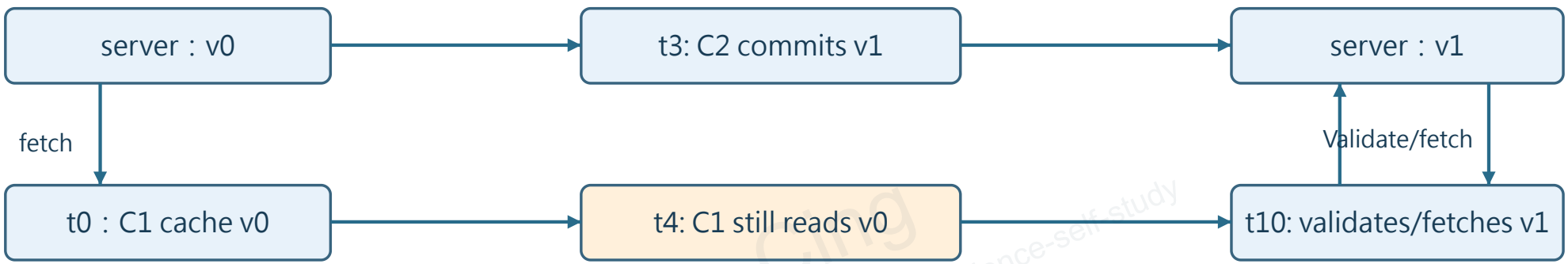
Per-read 1 ms validation adds 100 ms, for 159 ms. Cache hits need not avoid network RTT. Larger cache units can improve read-ahead or waste bandwidth on a one-byte read and reduce resident-block counts at fixed capacity.

Memory cache is volatile;disk cache retains durable bytes but needs metadata recovery/freshness checks. Servers cache too;server RAM acceptance is not stable storage. Write-through/back specify propagation timing, while the next layer's return defines durability separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cache consistency timelines

Server versions and client cache visibility have different timelines



Periodic checks permit staleness. Strong callback/lease models recall conflicting read rights before write completion.

Client-initiated validation: C1 reads/validates v0 at 0; C2 completes v1 at 3; C1 validating next at 10 can still return v0 at 4. This is not latest-completed-write visibility on each read. Open-time validation needs within-open semantics; every-access validation costs network and still needs coordination with the actual read.

Cache consistency timelines (continued)

Server-initiated coherence tracks cache/read/write rights and invalidates/recalls them, or disables cache on conflict. A strong model requires confirmation that conflicting read permissions are revoked/invalidated or their controlled leases expired before write completion. During partition, unlimited old-client reads cannot coexist with an immediate-visibility claim.

AFS/OpenAFS callbacks avoid checking every access, but do not rewrite application-owned bytes already read. Save/close and cache-manager refresh affect observations. Session, close-to-open, and stronger DFS guarantees are not synonyms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFSv3 state and write stability

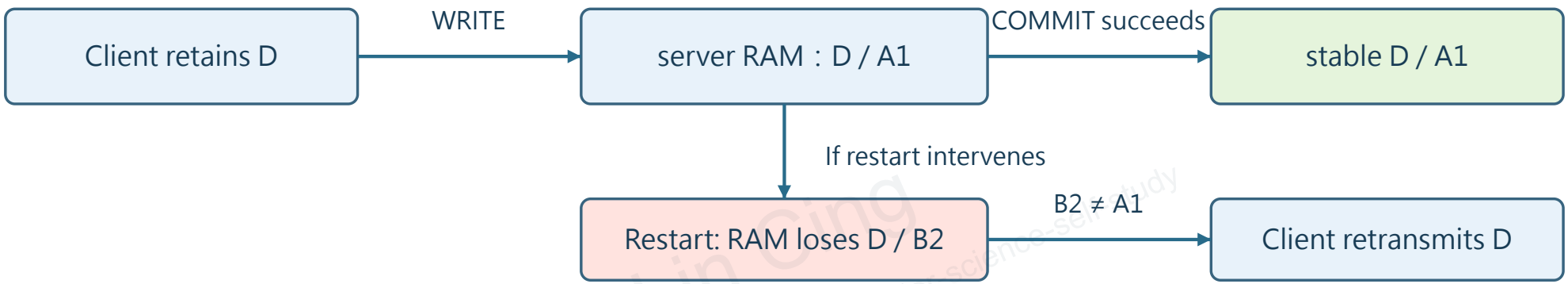
RFC 1813 READ/WRITE carry filehandle, offset, count rather than relying on equivalent server OPEN sessions. Server caches/duplicate-request state and separate lock protocols can still exist. Authentication, permissions, and handle lifetime remain relevant.

WRITE requests/replies distinguish UNSTABLE, DATA_SYNC, FILE_SYNC. UNSTABLE success is not durable data. Clients requiring later COMMIT/verifier checks retain retransmittable bytes. Weak cache-consistency attributes assist change detection without proving global linearizability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Restart after an unstable write

NFSv3: normal and restart paths after one successful WRITE



After UNSTABLE, retain data; a successful COMMIT requires a verifier check.
A retransmitted WRITE can confirm the bytes actually written at the required stability.

Client sends 4 KiB D at offset 0, UNSTABLE. Server receives into RAM and replies UNSTABLE, verifier A1; client retains D. Normal COMMIT succeeds with A1, establishing protocol-stable storage and permitting release of that retransmission obligation.

Restart after an unstable write (continued)

If restart loses volatile D, new instance uses B2. A COMMIT/WRITE returning B2≠A1 cannot certify the old data. Retransmit unconfirmed D and obtain the required stable result. COMMIT success without verifier comparison is insufficient.

This requires retained client bytes, correct server stability/verifier/range behavior, and does not give arbitrary concurrent writers application transactions. Durability, visibility, and RPC deduplication differ.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One remote operation across layers

Service name→DNS address→route→ARP→link/IP/transport→RPC ID→authorization/durable commit→client cache. Frame delivery does not prove RPC completion;TCP bytes do not prove one business execution;NFS UNSTABLE does not prove persistence.

Transparency offers a consistent interface inside stated boundaries without eliminating latency, partial failures, copies, or external effects. Insufficient observations require unknown/suspected rather than treating timeout as logical proof.

Split serial/parallel/coordination cost. Track bits/bytes, headers, RTT, windows. Draw reliability fault domains. Link RPC IDs to atomic effects;separate DFS names, identities, locations, visibility, durability. General guarantees depend on transitions:unbounded-delay timeout cannot prove crash, fencing needs resource-side epoch enforcement, and deduplication needs sufficient retention. Exercises include counterexamples, normal paths, and restart states.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One remote operation across layers (continued)

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 17. RFC 1034, RFC 826, RFC 1813. Hadoop 1.2.1 HDFS. OpenAFS.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study